GAME TEST REPORT

Dangrondropa Mines

Pre-Certification Self-Assessment

PASSED WITH NOTES

Report Number	`BS-SPINNY-20260801-90D9A939`
Date of Report	2026-08-01
Issued By	BlueStuff Game Studio · Automated Test Engine
Report Recipient	6a43beed9eae7b49adcdc60e
Engine Under Test	`spinny`
Standard Tested Against	GLI-19 v3.0 — Interactive Gaming Systems
Classification	Pre-certification self-assessment (Non-Jurisdictional)
Software Fingerprint	`sha256:028dfd9d728ae055205da993364b2b6f153d719a…` · 30 files
Result Summary	3 passed · 0 failed · 0 warnings · 1 manual · 0 source findings

Disclaimer. This is an automated pre-certification self-assessment produced by BlueStuff Game Studio's own test engine, published unmodified — including any failures. It is tested *against* the GLI-19 v3.0 technical standard but is **not** an accredited-laboratory certification and does not imply affiliation with, or endorsement by, Gaming Laboratories International. Final certification requires an independent accredited testing laboratory.

What this report is

This is an independent, automated fairness test of **Dangrondropa Mines**. A test engine played the game millions of times and measured whether it behaves fairly: is it truly random, does it pay back what it should, and can anyone break it? The results below are in plain language, and **every result links to the raw evidence** behind it — so nothing here has to be taken on trust.

1. Summary — what we found

Every hard requirement that ran passed. Some results are provisional or need human review.

All the evidence for this report: open the evidence folder.

Evaluator's Assessment

The reasoned opinion of the automated evaluator, based strictly on the measured results in this report.

GLI-19 Compliance Evaluation — Spinny (ea4b051c-ca04-410d-98d5-fc52a59a4872)

Run ID: `90d9a939-9935-4be8-a03d-2c52a2ac9531` | **Seed:** `gli-1785595944440` | **Date:** 2026-08-01

Overall Determination: Inconclusive — Not Fully Tested

The RNG subsystem is statistically sound, but the game cannot receive a compliance determination under GLI-19 because the payout engine produced zero payouts across 10,000,000 spins. No RTP can be certified, and §4.7.1 is therefore unresolved. A game without measurable game math is not a certifiable game. Until payout logic is implemented and produces verifiable results, this engine must not be offered to real-money players.

RNG Subsystem — Clauses 3.2.2, 3.2.3, 3.2.4: PASS

All seven statistical tests mandated by §3.2.2 were applied across five independent replicates at 10M spins per replicate. The Holm–Bonferroni-corrected family-wise error rate held at 99% confidence; no test was rejected. The Fisher-combined p-value across all seven tests was **0.1047** — comfortably above the 0.01 threshold. Uniformity (§3.2.3) was confirmed on both the float (32-bin, p = 0.0119) and integer (64-range, p = 0.100) paths. Independence (§3.2.4) was confirmed via overlaps, serial-correlation, and interplay-correlation (Fisher combined p = 0.394). These results are well-powered and credible. The shared crypto-backed RNG passes with no reservations.

Clause 4.7.1 (RTP) — MANUAL: Blocking Issue

The engine returned **no payouts** over 10,000,000 spins. The audit engine explicitly flagged this as a stub or non-payout engine and marked the result `MANUAL` — meaning a human must resolve this before any automated certification can proceed. This is the single most consequential finding in this report.

For real players and operators, this means:
- **There is no verifiable RTP.** Players would have no basis to know what return to expect.
- **Regulatory submission is impossible.** GLI-19 §4.7.1 requires a measured and certified RTP range. A zero-payout engine cannot satisfy this.
- **Operator liability is real.** Deploying a game that cannot demonstrate compliant RTP exposes the operator to regulatory sanction in every licensed jurisdiction.

Adversarial Testing — No Findings

No adversarial findings were returned. This is noted, but given that the payout engine appears to be a stub, the absence of runtime attack surfaces (double-spend, cash-out-after-settlement, concurrent bets) is expected rather than reassuring — there is nothing to exploit because the game does not yet pay out.

Recommendations (Prioritised)

1. **[Critical — Blocking]** Implement and register the payout/game-math engine. The `spinny` backend must produce actual spin outcomes with multiplier payouts before any compliance run is meaningful. Re-run §4.7.1 after implementation.
2. **[High]** Once payout logic is live, run RTP measurement at minimum 10M spins per configured multiplier table variant. The target RTP and its tolerance band must be defined and documented before testing begins.
3. **[Medium]** Re-run adversarial probes against the live payout backend. Runtime probes (concurrent bets, cash-out timing, session fixation) were not exercisable in this run and remain untested.
4. **[Low — Informational]** The RNG results are strong and require no remediation. These results can be carried forward to the next run provided the seed and engine version remain unchanged; they do not need to be repeated.

> **Staged for human review.** This report is a preliminary evaluation only. The `MANUAL` verdict on §4.7.1 means no automated pass/fail determination can be issued. This report should not be published to the public compliance repository until the payout engine is implemented, re-tested, and a human reviewer at NEGroup has confirmed the updated results.

2. Results at a glance

Each row is one thing we checked, in plain terms, with a link to the evidence behind it.

What we checked	Result	Based on	Evidence
Randomness quality	✅ **Passed**	10 million rounds	View
Even distribution	✅ **Passed**	10 million rounds	View
Independence of rounds	✅ **Passed**	2 million rounds	View
Payout fairness (RTP)	🔎 **Needs human review**	10 million rounds	View
Attempts to break the game	**Nothing found**	source + probes	View

3. What we tested, in plain terms

Randomness quality ✅ Passed

What we checked. We looked at whether the random-number generator behaves like true randomness, with no pattern a player or the operator could predict or exploit.

What we found. Shared RNG source (crypto-backed, used by every engine on this template). float() scaled to 32 bins, 5 independent draws combined by Fisher's method. 7 of the 7 tests named in §3.2.2 applied. None rejected at the 99% family-wise confidence level (Holm–Bonferroni; Fisher combined p = 1.047e-1).

Why it matters. If outcomes were predictable, the game would not be fair to either side. (GLI-19 3.2.2) · see the full evidence

Even distribution ✅ Passed

What we checked. We looked at whether every possible outcome comes up as often as it should — none too often, none too rarely.

What we found. RNG output is uniformly distributed (float 32-bin: Fisher chi-square = 22.69, p = 0.0119; int 64-range: p = 0.1000).

Why it matters. A fair game needs every symbol or result to appear at its intended rate. (GLI-19 3.2.3) · see the full evidence

Independence of rounds ✅ Passed

What we checked. We looked at whether each round stands on its own, with earlier results never nudging later ones.

What we found. Independence of the shared RNG (serial, overlaps, interplay) and of each engine's per-round outcomes (none playable). 3 of the 7 tests named in §3.2.2 applied. None rejected at the 99% family-wise confidence level (Holm–Bonferroni; Fisher combined p = 3.942e-1).

Why it matters. Past spins must not influence future spins — there are no hidden streaks the game steers. (GLI-19 3.2.4) · see the full evidence

Payout fairness (RTP) 🔎 Needs human review

What we checked. We looked at whether, over millions of rounds, the game returns the share of stakes it is designed to (its Return To Player).

What we found. Return-to-player measured per registered engine by driving play() with the template RNG: • example: no payouts observed over 10,000,000 spins — stub or non-payout engine; nothing to certify. No registered engine produced payouts — this backend has no certifiable game math yet. (1 non-payout/stub engine(s) noted above.)

Why it matters. This is the headline promise to players — that the game pays back what it advertises. (GLI-19 4.7.1) · see the full evidence

4. Attempts to break the game

Beyond the maths, we probed the game for the ways games actually get exploited — unfair logic, advertised prizes that can't be won, and fairness claims with nothing behind them. **None of these probes found an issue in this run.**

Some live-system attacks (for example placing the same bet twice at once, or cashing out after a round is settled) can only be tested against a running server; where those were not run against a live instance they are out of scope for this report rather than assumed safe.

5. The live deployed game

We also try to test the **actual game running live on the server** by playing real rounds through it. For this run that could not be done:

The deployed backend at https://smokestack-spinny-backend-stg.bluestufftech.com was not reachable (no /health), so live testing was skipped.

6. The evidence — and how to check it yourself

Every result above links to a file in the evidence folder for this report. Each file carries the exact numbers, the random seed, and the command that produced them.

Evidence file	What it contains
`3.2.2.json`	Full statistics, seed and reproduction command behind the 3.2.2 result.
`3.2.3.json`	Full statistics, seed and reproduction command behind the 3.2.3 result.
`3.2.4.json`	Full statistics, seed and reproduction command behind the 3.2.4 result.
`4.7.1.json`	Full statistics, seed and reproduction command behind the 4.7.1 result.
`adversarial-findings.json`	Every "attempt to break the game" probe and what it found (empty if none).
`audit.full.json`	The entire machine-readable audit: every verdict, every per-test statistic, the collective evaluation, seeds and commands.
`live-tests.json`	Runtime checks run against the live deployed container — double-spend, session/validation, and the deployed-vs-certified RTP cross-check, with the real rounds played.
`checksums.json`	SHA-256 of each of the 30 tested source files, plus the aggregate fingerprint this report is bound to.
`README.md`	Plain-language index of this evidence folder and how to verify results.

Reproduce the whole run yourself:

```
node src/cli.ts --engine=spinny --repo=/opt/ne/sportsbook/mcp/mcp-server/.repo-cache/games/bluestuff-io__smokestack-original-games/BackendEngines/spinny --samples=2000000 --spins=10000000 --replicates=5 --seed=gli-1785595944440
```

The test engine has no external dependencies and no AI in its numeric path — every statistic is computed by reviewable, in-tree code. Re-running the command with the same seed reproduces the same numbers exactly.

7. Software fingerprint

We took a SHA-256 checksum of every one of the 30 source files tested. The single fingerprint below is a hash over all of them — change any tested file and it changes too, so this report is locked to exactly the code that was tested.

Aggregate fingerprint: `sha256:028dfd9d728ae055205da993364b2b6f153d719a0737f5f688347eb94eba049f`

Full per-file checksums: `checksums.json` .

8. Test Environment

Test engine	SportsITDev/gli-audit-engine
Runtime	Node v24.18.0
Seed	`gli-1785595944440`
Started	2026-08-01T14:52:24.546Z
Finished	2026-08-01T14:53:18.345Z
Generated by	gli-audit-engine game-tester agent